

AI monks.io · [Follow publication](#)

Multi-agent System Design Patterns From Scratch In Python | ReAct Agents

25 min read · May 20, 2025



Prince Krampah

[Follow](#)

Listen



Share

Hello there guys, welcome back to another exciting article in our series on building AI agentic patterns from scratch using plain Python. This is the third article in this series and in this article, we'll be looking into how to build a Reasoning and Acting agent (a ReAct agent).

In complex AI workflows and multi-agent architectures, one common thing you want your AI agents to do is to first reason over a query before taking actions to use a tool. This was the very reason for the need and introduction of ReAct agents, to tackle exactly this need.

In this article, we'll dive into how to implement one of these ReAct agents from scratch using plain Python code.

Let's get started!

What Is A ReAct Agent

The React pattern is part of a broader agentic pattern called the `planning pattern`. ReAct is a pattern that combines Reasoning and Acting. It is a way to make an agent more flexible and adaptable. The reasoning part is the agent's ability to think about a problem and the acting part is the agent's ability to take action using the tools provided to it.

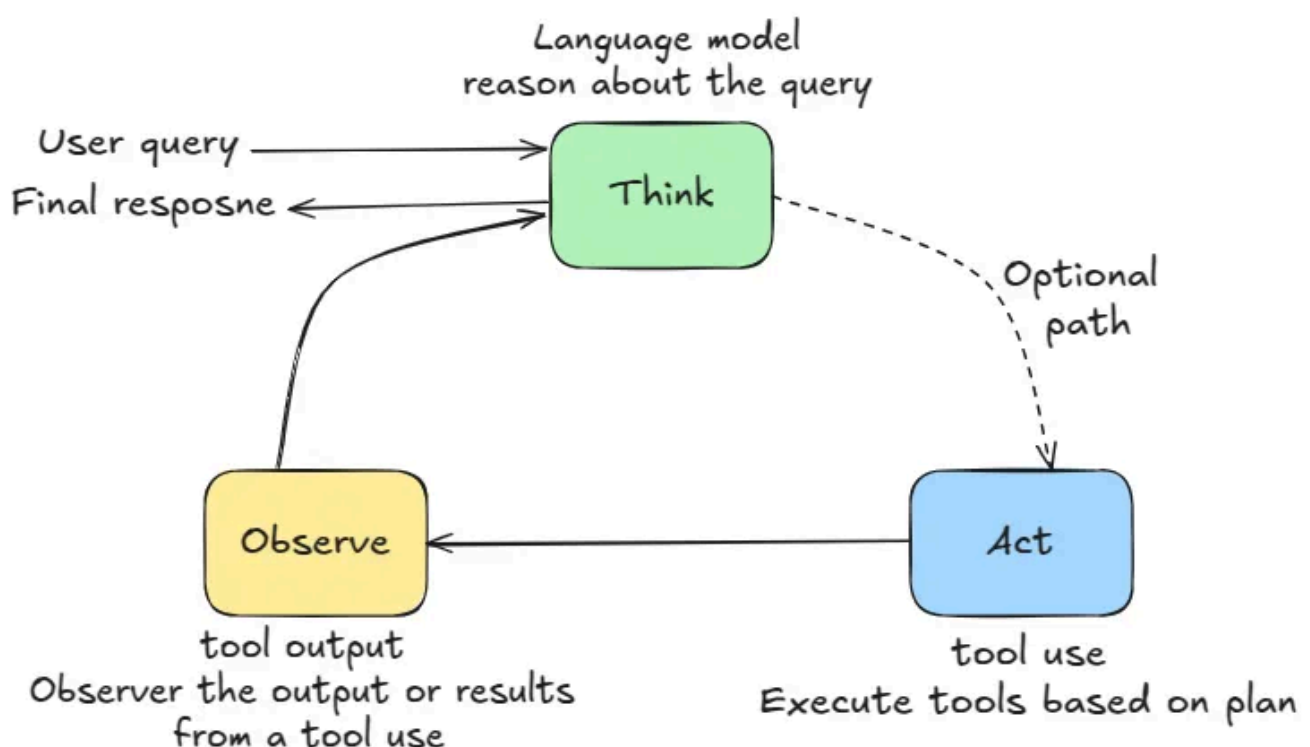
In the last session, we created a tool calling agent that could call a tool to calculate the area of a rectangle. In this session, we will create a ReAct agent that will be able to utilize some of this tool calling logic to take actions.



Image By Code With Prince

How A ReAct Agent Works | ReAct Agent Architecture

The ReAct agent operates through a structured workflow:



Reasoning Phase

When a user submits a query, the agent first analyzes it thoroughly using its language model capabilities. This involves understanding the request, identifying key components, and determining what's needed to respond effectively.

Decision Point

After comprehending the query, the agent reaches a critical decision:

- Respond directly using natural language if the answer is within its knowledge
- OR utilize one of its available tools if specialized processing is required

Tool Execution

If a tool is needed, the agent:

- Selects the appropriate tool from its available options
- Formulates the correct parameters to pass to that tool
- Outputs this as a structured tool call
- The system extracts this output and executes the actual tool function in Python
- The result is returned to the agent as an observation

Response Generation

With the tool results in hand, the agent:

- Formulates a coherent natural language response if the task is complete
- OR determines another tool is needed and repeats the process

This creates a powerful loop of Think → Act → Observe → Respond that allows the agent to handle multi-step tasks requiring different tools and reasoning stages. The beauty of this pattern is its flexibility — it can chain multiple tools together to solve complex problems while maintaining a natural conversation flow with the user.

Getting Started

To get started, let's bring in some of the code from the last session in here so we can make use of it.

```

from typing import Callable, Dict, Any
import json

def extract_function_metadata(function: Callable) -> Dict[str, Any]:
    """
    Creates a metadata dictionary from a function's signature.

    Parameters:
        function: The target function to analyze

    Returns:
        A dictionary containing the function's name, documentation, and parameters
    """
    # Initialize the basic structure
    metadata = {
        "name": function.__name__,
        "description": function.__doc__,
        "parameters": {"properties": {}}
    }

    # Extract parameter types (excluding return annotation)
    parameter_types = {
        param_name: {"type": param_type.__name__}
        for param_name, param_type in function.__annotations__.items()
        if param_name != "return"
    }

    # Add parameters to metadata
    metadata["parameters"]["properties"] = parameter_types
    return metadata

def convert_argument_types(tool_invocation: Dict[str, Any], function_spec: Dict[str, Any]) -> Dict[str, Any]:
    """
    Ensures all arguments match their expected types according to the function specification.

    Parameters:
        tool_invocation: Dictionary with the tool name and arguments
        function_spec: Dictionary containing the expected parameter types

    Returns:
        Updated tool invocation with correctly typed arguments
    """
    expected_params = function_spec["parameters"]["properties"]

    # Type conversion mapping
    type_converters = {
        "int": int,
        "str": str,
        "bool": bool,
        "float": float
    }

```

```

# Convert each argument to its expected type if needed
for arg_name, arg_value in tool_invocation["arguments"].items():
    target_type = expected_params[arg_name].get("type")
    if not isinstance(arg_value, type_converters[target_type]):
        tool_invocation["arguments"][arg_name] = type_converters[target_type](arg_value)

return tool_invocation

class Tool:
    """
    Wrapper class that encapsulates a function as a callable tool.

    Attributes:
        name: Tool identifier
        function: The underlying function
        specification: JSON-formatted function metadata
    """
    def __init__(self, name: str, function: Callable, specification: str):
        self.name = name
        self.function = function
        self.specification = specification

    def __str__(self):
        return self.specification

    def execute(self, **kwargs):
        """
        Runs the wrapped function with the provided arguments.

        Parameters:
            **kwargs: Arguments to pass to the function

        Returns:
            The function's result
        """
        return self.function(**kwargs)

def tool(function: Callable) -> Tool:
    """
    Decorator that transforms a regular function into a Tool instance.

    Parameters:
        function: The function to convert into a tool

    Returns:
        A fully configured Tool object
    """
    def create_tool():
        metadata = extract_function_metadata(function)
        return Tool(
            name=metadata.get("name"),
            function=function,

```

```

        specification=json.dumps(metadata)
    )

    return create_tool()

```

If you do not understand what this code does, it simply makes a Python function as a tool that our AI agents can use through the `tool` decorator function we have above.

A decorator is a callable that takes in a callable as an input and returns a callable as an output. To understand this even better, I urge you to take a look into the [last article](#) I posted on this.

From the last article as well, we created the `ToolCallingAgent` class as well. We'll also bring in the code here, bringing in this code should give you an idea of what we'll use it for, right? Anyway, here is the code:

```

import json
import re
from typing import List, Dict, Any, Callable
from colorama import init, Fore, Style

# Initialize colorama
init(autoreset=True)

class ToolCallingAgent:
    """
    An agent that integrates language models with function-calling tools.

    This class manages the interaction between a language model and a set of tools,
    allowing the model to decide when to call functions and processing the results.
    """

    def __init__(self, llm: Callable, system_prompt: str, tools: List[Any]):
        """
        Initialize an AI agent with a language model and tools.

        Args:
            llm: Function that takes a prompt string and returns a response
            system_prompt: Instructions for guiding the language model's behavior
            tools: List of Tool objects that the agent can use
        """
        self.model = llm
        self.system_prompt = system_prompt
        self.tools = {tool.name: tool for tool in tools}

```

```

self.conversation_history = []
print(f"{Fore.GREEN}ToolCallingAgent initialized with {len(tools)} tools")

def format_tools_for_prompt(self) -> str:
    """Format all available tools into a format the language model can understand"""
    tools_json = []
    for tool in self.tools.values():
        try:
            # Use the specification provided by the @tool decorator
            tools_json.append(json.loads(tool.specification))
        except (json.JSONDecodeError, AttributeError):
            # Fallback for tools without proper specification
            tools_json.append({
                "name": tool.name,
                "description": getattr(tool, "description", "No description provided"),
                "parameters": {"properties": {}}
            })

    print(f"{Fore.CYAN}Formatted {len(tools_json)} tools for LLM prompt{Style.RESET_ALL}")
    return f"<tools>\n{json.dumps(tools_json, indent=2)}\n</tools>"

def extract_tool_calls(self, response: str) -> List[Dict[str, Any]]:
    """
    Parse the language model's response to extract tool call requests.

    Args:
        response: The text response from the language model

    Returns:
        A list of tool call dictionaries with 'name' and 'arguments' keys
    """
    tool_calls = []
    pattern = r"<tool_call>(.*?)</tool_call>"
    matches = re.findall(pattern, response, re.DOTALL)

    for match in matches:
        try:
            tool_call = json.loads(match.strip())
            if "name" in tool_call and "arguments" in tool_call:
                tool_calls.append(tool_call)
        except json.JSONDecodeError:
            continue

    if tool_calls:
        print(f"{Fore.YELLOW}Extracted {len(tool_calls)} tool call(s) from response{Style.RESET_ALL}")
    else:
        print(f"{Fore.YELLOW}No tool calls found in LLM response{Style.RESET_ALL}")

    return tool_calls

def execute_tool(self, tool_call: Dict[str, Any]) -> Any:
    """
    Execute a tool based on the model's request.
    """

```

```

Args:
    tool_call: Dictionary with 'name' and 'arguments' for the tool

Returns:
    The result from executing the tool
"""
tool_name = tool_call.get("name")
arguments = tool_call.get("arguments", {})

if tool_name not in self.tools:
    print(f"{Fore.RED}Error: Tool '{tool_name}' not found{Style.RESET_ALL}")
    return f"Error: Tool '{tool_name}' not found"

tool = self.tools[tool_name]
print(f"{Fore.MAGENTA}Executing tool: {tool_name} with arguments: {json.dumps(arguments)}")

# Validate and convert argument types using the tool's specification
try:
    if hasattr(tool, "specification"):
        tool_spec = json.loads(tool.specification)
        validated_args = self.convert_argument_types(
            {"arguments": arguments},
            tool_spec
        )["arguments"]
        arguments = validated_args
        print(f"{Fore.BLUE}Arguments validated and converted to appropriate types{Style.RESET_ALL}")
except (json.JSONDecodeError, AttributeError, KeyError) as e:
    print(f"{Fore.RED}Error validating arguments: {str(e)}{Style.RESET_ALL}")
    # Continue with original arguments if validation fails
    pass

try:
    # Handle execution based on the tool interface
    # First try the execute method for tools created with the @tool decorator
    if hasattr(tool, "execute"):
        print(f"{Fore.GREEN}Calling tool.execute() method{Style.RESET_ALL}")
        return tool.execute(**arguments)
    # Then try the function attribute which is used by the @tool decorator
    elif hasattr(tool, "function"):
        print(f"{Fore.GREEN}Calling tool.function() method{Style.RESET_ALL}")
        return tool.function(**arguments)
    # Fall back to run method
    elif hasattr(tool, "run"):
        print(f"{Fore.GREEN}Calling tool.run() method{Style.RESET_ALL}")
        return tool.run(**arguments)
    # Last resort: call the tool directly if it's callable
    elif callable(tool):
        print(f"{Fore.GREEN}Calling tool directly{Style.RESET_ALL}")
        return tool(**arguments)
    else:
        print(f"{Fore.RED}Error: Tool '{tool_name}' is not callable{Style.RESET_ALL}")
        return f"Error: Tool '{tool_name}' is not callable"

```



```

except Exception as e:
    print(f"{Fore.RED}Error executing {tool_name}: {str(e)}{Style.RESET_ALL}")
    return f"Error executing {tool_name}: {str(e)}"

def convert_argument_types(self, tool_call: Dict[str, Any], tool_spec: Dict[str, Any]) -> Dict[str, Any]:
    """
    Convert arguments to their expected types based on tool specification.

    Args:
        tool_call: Dictionary containing arguments to convert
        tool_spec: Tool specification with expected types

    Returns:
        Updated tool call with properly typed arguments
    """
    if "parameters" not in tool_spec or "properties" not in tool_spec["parameters"]:
        return tool_call

    properties = tool_spec["parameters"]["properties"]

    # Standard type converters
    type_mapping = {
        "int": int,
        "str": str,
        "bool": bool,
        "float": float,
        "integer": int,
        "string": str,
        "boolean": bool,
        "number": float
    }

    for arg_name, arg_value in tool_call["arguments"].items():
        if arg_name in properties and "type" in properties[arg_name]:
            expected_type = properties[arg_name]["type"]

            if expected_type in type_mapping:
                converter = type_mapping[expected_type]
                try:
                    # Only convert if types don't match
                    if not isinstance(arg_value, converter):
                        print(f"{Fore.BLUE}Converting argument '{arg_name}' to {expected_type}")
                        tool_call["arguments"][arg_name] = converter(arg_value)
                except (ValueError, TypeError) as e:
                    print(f"{Fore.RED}Type conversion error for '{arg_name}': {str(e)}")
                    # Keep original value if conversion fails
                    pass

    return tool_call

def run(self, user_input: str) -> str:
    print(f"{Fore.WHITE}{Style.BRIGHT}=== Starting agent run with user input: {user_input}")
    # Add user input to conversation history

```

```

self.conversation_history.append({"role": "user", "content": user_input

# Build the messages list for OpenAI API
messages = [
    {"role": "system", "content": self.system_prompt + "\n\n" + self.fc
]

# Add conversation history
for message in self.conversation_history:
    messages.append({"role": message["role"], "content": message["conte

# Get response from language model
print(f"{Fore.CYAN}Calling language model...{Style.RESET_ALL}")
response = self.model(model="gpt-4o", messages=messages)
model_response = response.choices[0].message.content
print(f"{Fore.CYAN}Received response from language model ({len(model_re

# Extract tool calls from response
tool_calls = self.extract_tool_calls(model_response)

# If no tool calls, return the response directly
if not tool_calls:
    print(f"{Fore.GREEN}No tool calls needed. Returning response.{Style
    final_response = model_response
else:
    # Execute tools and collect results
    tool_results = []
    for i, tool_call in enumerate(tool_calls):
        print(f"{Fore.MAGENTA}{Style.BRIGHT}Executing tool call {i+1}/{
        result = self.execute_tool(tool_call)
        tool_results.append({
            "tool": tool_call.get("name"),
            "arguments": tool_call.get("arguments"),
            "result": result
        })

    # Format tool results
    results_text = "Tool results:\n"
    for res in tool_results:
        result_str = str(res["result"])
        if isinstance(res["result"], (dict, list)):
            try:
                result_str = json.dumps(res["result"], indent=2)
            except:
                pass

        results_text += f"- {res['tool']} {json.dumps(res['arguments'])}

    print(f"{Fore.BLUE}Formatted tool results{Style.RESET_ALL}")

# Create a new message with original response and tool results
messages.append({"role": "assistant", "content": model_response})
messages.append({"role": "user", "content": results_text})

```

```

        # Get final response from language model with tool results
        print(f"{Fore.CYAN}Calling language model with tool results...{Style.RESET_ALL}")
        final_response_obj = self.model(model="gpt-4o", messages=messages)
        final_response = final_response_obj.choices[0].message.content
        print(f"{Fore.CYAN}Received final response from language model ({len(final_response)} characters){Style.RESET_ALL}")

    # Add final response to conversation history
    self.conversation_history.append({
        "role": "assistant",
        "content": final_response
    })

    print(f"{Fore.WHITE}{Style.BRIGHT}=== Agent run completed ==={Style.RESET_ALL}")
    return final_response

def reset_conversation(self):
    """Clear the conversation history."""
    print(f"{Fore.GREEN}Conversation history reset{Style.RESET_ALL}")
    self.conversation_history = []

```

Again, if you are finding it difficult to understand this code above, a strongly urge you to take a look into the [last article](#) on tool use.

Building The ReAct Agent

Once we have brought in the code from the previous article, we can begin to implement the ReAct agent pattern. Let's start by bringing in some imports we can make use of. We can then bring in some other imports we will need for this session and create the LLM model we'll be using.

```

from openai import OpenAI
from dotenv import load_dotenv
import os

load_dotenv()

```

We can then load in our environment variables, make sure to have your API keys available in the `.env` file. Once this is set, we can proceed:

```
client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))
```

ReAct Agent System Prompt

Once we have our environment variables loaded in, we can move on to defining a system prompt for our ReAct agent, here is an example, again feel free to modify this to your own use cases.

```
REACT_SYSTEM_PROMPT = """
# ReAct Agent: Reasoning and Acting Framework

You are an AI assistant that follows the ReAct (Reasoning and Acting) framework

## How You Operate

1. **THOUGHT**: Carefully reason about the problem and determine if a tool call
2. **ACTION**: Execute tools based on your reasoning, following the proper form
3. **OBSERVATION**: Analyze the results to inform your final response

## Available Tools

You have access to the following functions to help users:

<tools>
__TOOLS__
</tools>

## Tool Calling Format

When you decide to use a tool, format your call exactly as follows:

<tool_call>
{"name": "function_name", "arguments": {"param1": "value1", "param2": "value2"}}
</tool_call>

For example, if you have a tool with this definition:

<tool_call>
{
  "name": "add_two_numbers",
  "description": "Used to add two numbers together",
  "parameters": {
    "properties": {
      "a": {
        "type": "int"
      },
      "b": {
```

```
        "type": "int"
    }
}
}
</tool_call>
```

Your tool call should look like:

```
<tool_call>
{"name": "add_two_numbers", "arguments": {"a": 1, "b": 2}}
</tool_call>
```

Interaction Flow

Follow this structured approach to every user request:

1. <thought>Your detailed reasoning about how to approach the problem</thought>
2. <tool_call>Your properly formatted tool call if needed</tool_call>

After your tool call, you will receive:

```
<observation>Results from the tool execution</observation>
```

Then provide your final answer:

```
<response>Your complete, helpful answer based on the tool results</response>
```

Example Interaction

```
<question>What's the sum of 10 and 20?</question>
```

```
<thought>To answer this question, I need to get the sum of 10 and 20. I should
<tool_call>{"name": "add_two_numbers", "arguments": {"a": 10, "b": 20}}</tool_c
```

```
[System provides tool result]
<observation>{"result": 30}</observation>
```

```
<response>The sum of 10 and 20 is 30.</response>
```

Important Guidelines

- Always begin with a <thought> tag to show your reasoning process
 - Carefully inspect the function signature before making a tool call
 - Pay special attention to parameter types and requirements
 - Don't make assumptions about parameter values
 - Use proper JSON syntax in your tool calls
 - If a query doesn't require tools, respond directly:
 <thought>This question doesn't require any tool use because...</thought>
 <response>Your helpful answer here...</response>
 - For complex tasks that require multiple tools, use one tool at a time
 - If you receive unclear results, think about alternative approaches
- """

Explanation of the ReAct Agent System Prompt

This system prompt creates an AI agent that follows the Reasoning and Acting (ReAct) framework. Here's a breakdown of how it works and how to customize it:

XML Tags

The prompt uses several custom XML tags to structure the agent's thought process and actions:

1. `<thought>...</thought>` — Contains the agent's reasoning process before taking any actions
2. `<tool_call>...</tool_call>` — Wraps the JSON for calling a specific function/tool
3. `<observation>...</observation>` — Contains the results returned from tool execution
4. `<response>...</response>` — Contains the agent's final answer to the user
5. `<tools>...</tools>` — Defines all available tools the agent can use

The `__TOOLS__` Placeholder

The `__TOOLS__` placeholder is where you'll inject the actual tool definitions before using the prompt. Here's how to replace it:

1. Generate JSON definitions for each tool you want to make available
2. Format each tool as a JSON object with `name`, `description`, and `parameters`
3. Replace `__TOOLS__` with the string representation of these tool definitions

For example, if you have two tools, you might replace it like this:

```
tools = [  
  {  
    "name": "calculate_area",  
    "description": "Calculate the area of a rectangle",  
    "parameters": {  
      "properties": {  
        "length": {"type": "float"},  
        "width": {"type": "float"}  
      }  
    }  
  },  
  {  
    "name": "get_current_weather",  
    "description": "Get the current weather for a location",
```

```

        "parameters": {
            "properties": {
                "location": {"type": "string"},
                "unit": {"type": "string"}
            }
        }
    }
]

# Convert tools to JSON string
tools_json = json.dumps(tools, indent=2)

# Replace __TOOLS__ in the prompt
final_prompt = REACT_SYSTEM_PROMPT.replace("__TOOLS__", tools_json)

```

Agent Workflow

The prompt creates an agent that:

1. Takes in a user query
2. Thinks through the problem (in `<thought>` tags)
3. If needed, calls a tool (in `<tool_call>` tags)
4. Receives results (in `<observation>` tags)
5. Provides a final answer (in `<response>` tags)

Get Prince Krampah's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email

Subscribe

☐ Remember me for faster sign in

This structured approach ensures transparency in the agent's reasoning process and makes debugging easier, as you can see exactly why the agent chose to call a specific tool and how it interpreted the results.

Creating Tools

We can now create some tools to use with our ReAct agent. We'll create two main tools to use:

1. Addition tool

2. A tool to calculate the area of a rectangle Let's create these tools!

Let's create these tools!

```
@tool
def add_two_numbers(a: int, b: int) -> int:
    """Used to add two numbers together"""
    return a + b

@tool
def calculate_area_of_rectangle(length: float, width: float) -> float:
    """Used to calculate the area of a rectangle"""
    return length * width
```

We can view the specification of the tools we created by calling the `.specification` attribute on the tool.

```
add_two_numbers.specification
```

```
'{"name": "add_two_numbers", "description": "Used to add two numbers together",
```

Now, let's get all the tool signatures or specifications and create a JSON string representation of them.

```
tools = [add_two_numbers, calculate_area_of_rectangle]
tools_mapping = {tool.name: tool for tool in tools}

tools_specifications = "".join([tool.specification for tool in tools])

print(tools_specifications)
```



```
{"name": "add_two_numbers", "description": "Used to add two numbers together",
```

We can now add this signature into our system prompt by replacing the `__TOOLS__` string with our `tools_specifications` variable we created above:

```
REACT_SYSTEM_PROMPT = REACT_SYSTEM_PROMPT.replace("__TOOLS__", tools_specifications)
print(REACT_SYSTEM_PROMPT)
```

Here is the system prompt with the tool specifications:

```
# ReAct Agent: Reasoning and Acting Framework
```

You are an AI assistant that follows the ReAct (Reasoning and Acting) framework

```
## How You Operate
```

1. **THOUGHT**: Carefully reason about the problem and determine if a tool call
2. **ACTION**: Execute tools based on your reasoning, following the proper format
3. **OBSERVATION**: Analyze the results to inform your final response

```
## Available Tools
```

You have access to the following functions to help users:

```
<tools>
```

```
{"name": "add_two_numbers", "description": "Used to add two numbers together",
</tools>
```

```
## Tool Calling Format
```

When you decide to use a tool, format your call exactly as follows:

```
<tool_call>
```

```
{"name": "function_name", "arguments": {"param1": "value1", "param2": "value2"}}
</tool_call>
```

For example, if you have a tool with this definition:

```
<tool_call>
{
  "name": "add_two_numbers",
  "description": "Used to add two numbers together",
  "parameters": {
    "properties": {
      "a": {
        "type": "int"
      },
      "b": {
        "type": "int"
      }
    }
  }
}
</tool_call>
```

Your tool call should look like:

```
<tool_call>
{"name": "add_two_numbers", "arguments": {"a": 1, "b": 2}}
</tool_call>
```

Interaction Flow

Follow this structured approach to every user request:

1. <thought>Your detailed reasoning about how to approach the problem</thought>
2. <tool_call>Your properly formatted tool call **if** needed</tool_call>

After your tool call, you will receive:

```
<observation>Results from the tool execution</observation>
```

Then provide your final answer:

```
<response>Your complete, helpful answer based on the tool results</response>
```

Example Interaction

```
<question>What's the sum of 10 and 20?</question>
```

```
<thought>To answer this question, I need to get the sum of 10 and 20. I should
<tool_call>{"name": "add_two_numbers", "arguments": {"a": 10, "b": 20}}</tool_c
```

```
[System provides tool result]
<observation>{"result": 30}</observation>
```

```
<response>The sum of 10 and 20 is 30.</response>
```

Important Guidelines

- Always begin with a <thought> tag to show your reasoning process

- Carefully inspect the function signature before making a tool call
- Pay special attention to parameter types and requirements
- Don't make assumptions about parameter values
- Use proper JSON syntax `in` your tool calls
- If a query doesn't require tools, respond directly:
`<thought>This question doesn't require any tool use because...</thought>`
`<response>Your helpful answer here...</response>`
- For complex tasks that require multiple tools, use one tool at a time
- If you receive unclear results, think about alternative approaches

First Testing

Now, let's get some initial testing going for this custom ReAct agent.

```
user_query = "The sum of 10 and 20 is the width of a rectangle that is 100 units high"

user_prompt = [
    {"role": "system", "content": REACT_SYSTEM_PROMPT},
    {"role": "user", "content": f"<question>{user_query}</question>"}
]
```

```
response = client.chat.completions.create(
    model="gpt-4o",
    messages=user_prompt,
)

formatted_response = response.choices[0].message.content

print(formatted_response)
```

```
<thought>To find the area of the rectangle, I first need to determine the width
<tool_call>{"name": "add_two_numbers", "arguments": {"a": 10, "b": 20}}</tool_call>
```

From the above code, we can see that our ReAct agent was able to understand the user query, think through it and suggested we use the `add_two_numbers` prompt to attempt to answer the question. Now that we have the LLM pointing out the right

tool to use and the correct parameters to specify, what next? Well, let's take a look at our diagram again:

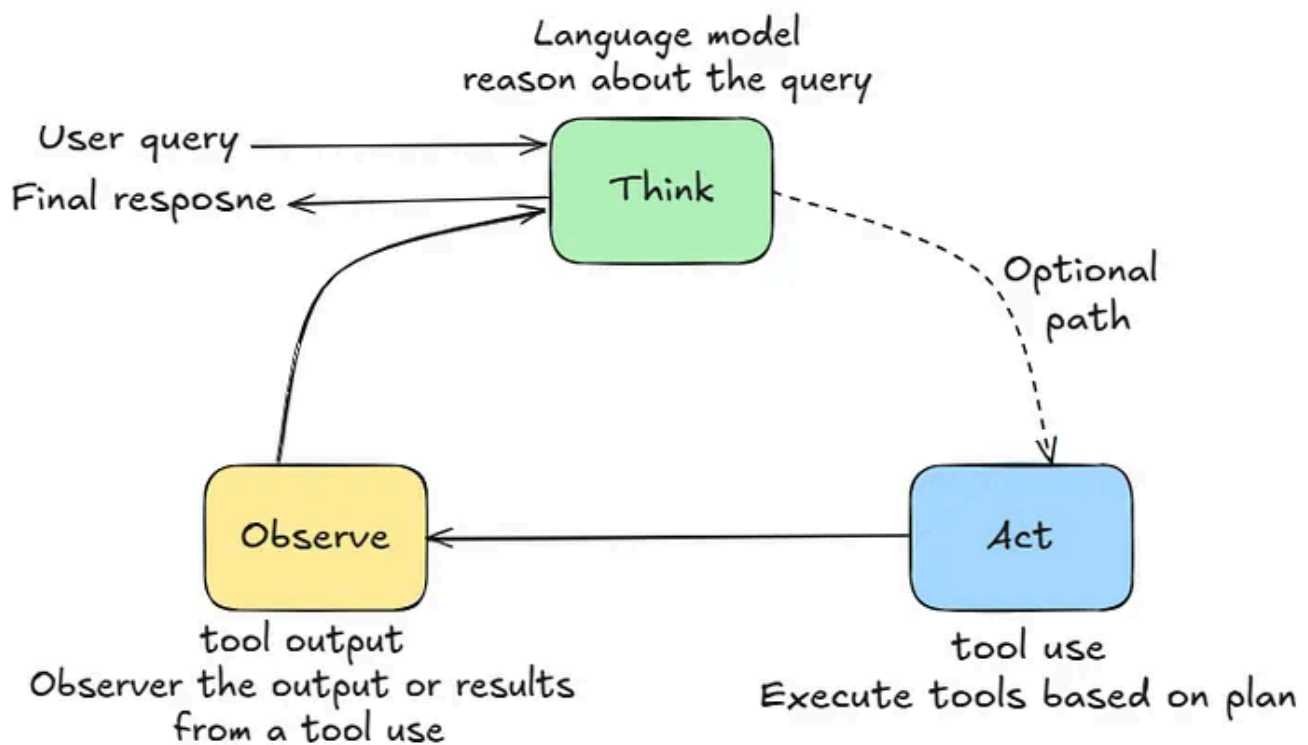


Image By Code With Prince

At this stage of our ReAct agent execution, our LLM has thought and suggested an action to take through the use of a tool. We have to execute the tool and return the output of this tool execution as an observation back to the LLM for it to generate a final response, in some cases, the LLM would suggest another tool call. This example has such a case, first, we need to find the sum of 10 and 20 then find the area of the rectangle. For now, let's execute the tool call the LLM suggested and return the response back to it.

```
def transform_response_to_dict(response_content: str) -> dict:
    """
    Extracts all <tool_call> JSON blocks from the response_content and returns
    Handles errors gracefully.
    """
    import re
    import json

    tool_calls = []
    try:
        # Find all <tool_call>...</tool_call> blocks
        matches = re.findall(r"<tool_call>\s*({.*?})\s*</tool_call>", response_content)
```

```

    if not matches:
        raise ValueError("No <tool_call> JSON found in response.")

    for match in matches:
        try:
            tool_calls.append(json.loads(match))
        except json.JSONDecodeError as e:
            tool_calls.append({
                "name": "JSONDecodeError",
                "message": f"Error decoding JSON: {str(e)}",
                "raw": match
            })
    return tool_calls
except Exception as e:
    return [{
        "name": type(e).__name__,
        "message": str(e),
        "stack": None
    }]

```

```

transformed_response = transform_response_to_dict(formatted_response)
print(transformed_response)

```

This function is simply going to return to us a Python dictionary, getting rid of all the unwanted strings like `<tool>`, more on this in the last article. Anyway, we can see the output of this:

```
[{'name': 'add_two_numbers', 'arguments': {'a': 10, 'b': 20}}]
```

From this, we can now go ahead and call the `add_two_numbers` function and return the results:

```

tool_result = ""

for tool_call in transformed_response:
    tool_name = tool_call.get("name")
    tool = tools_mapping.get(tool_name)
    if tool:
        tool_result = tool.execute(**tool_call.get("arguments", {}))

```

```
print(f"Tool {tool_name} executed with result: {tool_result}")
else:
    print(f"Tool {tool_name} not found in tools_mapping")
```

Tool add_two_numbers executed with result: 30

We then send the tool results back to the LLM as part of the prompts

```
user_prompt.append({"role": "user", "content": f"<observation>{tool_result}</ob
print(user_prompt)
```

```
[{'role': 'system',
  'content': '\n# ReAct Agent: Reasoning and Acting Framework\n\nYou are an AI
{'role': 'user',
  'content': '<question>The sum of 10 and 20 is the width of a rectangle that i
{'role': 'user', 'content': '<observation>""</observation>'},
{'role': 'user', 'content': '<observation>""</observation>'},
{'role': 'user', 'content': '<observation>30</observation>'}]
```

We can then call the LLM again with this new prompt as the one above:

```
response = client.chat.completions.create(
    model="gpt-4o",
    messages=user_prompt,
)
```

```
formatted_response = response.choices[0].message.content
print(formatted_response)
```

```
<response>The sum of 10 and 20 is 30, which is the width of the rectangle. To f  
  
<thought>The width of the rectangle is 30 units and the length is 100 units. I  
<tool_call>{"name": "calculate_area_of_rectangle", "arguments": {"length": 100,
```

From the output above, we can see the LLM now wants to calculate the area of the rectangle. This is since the tool call has been executed and the result has been returned for the addition of 10 and 20. Powered with this information, the LLM can now calculate the area of the rectangle. This is the Observation bit of a ReAct agent. Action, Observation, Thought, Response if no further tool calls are needed. At this point we simply repeat the same process over again:

```
transformed_response = transform_response_to_dict(formatted_response)  
print(transformed_response)
```

```
[{'name': 'calculate_area_of_rectangle', 'arguments': {'length': 100, 'width':
```

```
tool_result = ""  
  
for tool_call in transformed_response:  
    tool_name = tool_call.get("name")  
    tool = tools_mapping.get(tool_name)  
    if tool:  
        tool_result = tool.execute(**tool_call.get("arguments", {}))  
        print(f"Tool {tool_name} executed with result: {tool_result}")  
    else:  
        print(f"Tool {tool_name} not found in tools_mapping")
```

```
Tool calculate_area_of_rectangle executed with result: 3000
```

```
user_prompt.append({"role": "user", "content": f"<observation>{tool_result}</ob
```

```
response = client.chat.completions.create(  
    model="gpt-4o",  
    messages=user_prompt,  
)
```

```
formatted_response = response.choices[0].message.content  
print(formatted_response)
```

```
<response>The area of the rectangle with a length of 100 units and a width of 3
```

Okay, we finally got the correct response! In some cases, you might need to play around with your prompts to get this working well. You can see how this can be so iterative, repeating the same code over and over again in a LOOP. The key word here is LOOP, we can put this in a loop and let the program run, the LLM can make as many tool calls as it wants, the loop will handle it. I have created a Python class to help me do just that.

Project Folder Structure

Here is the project structure I am going to use to implement this logic. For now focus on only the ReAct agent piece:

```
.  
├── agent-patterns  
│   ├── multi_agents  
│   │   └── notebooks  
│   │       ├── lesson_01.ipynb  
│   │       ├── parallel_workflow.png  
│   │       └── sequential_workflow.png  
│   ├── ReAct_agent  
│   │   └── notebooks  
│   │       └── lesson_01.ipynb  
└── reflection_agent
```



```

|   |   └─ notebooks
|   |       └─ lesson_01.ipynb
|   └─ tool_calling
|       └─ notebooks
|           └─ lesson_01.ipynb
└─ custom_react_agent_with_langgraph.ipynb
└─ images
    └─ react_agent.png
    └─ tool_calling_agent.png
└─ python_classes
    └─ ReAct_agent
        └─ agent.py
        └─ __init__.py
        └─ __pycache__
            └─ agent.cpython-313.pyc
            └─ __init__.cpython-313.pyc
    └─ reflection_agent
        └─ __init__.py
        └─ main.py
        └─ __pycache__
            └─ __init__.cpython-311.pyc
            └─ __init__.cpython-313.pyc
            └─ main.cpython-311.pyc
            └─ main.cpython-313.pyc
    └─ tool_calling_agent
        └─ agent.py
        └─ decorators.py
        └─ __init__.py
        └─ __pycache__
            └─ agent.cpython-312.pyc
            └─ agent.cpython-313.pyc
            └─ decorators.cpython-312.pyc
            └─ decorators.cpython-313.pyc
            └─ __init__.cpython-312.pyc
            └─ __init__.cpython-313.pyc
└─ tests
    └─ __init__.py
    └─ react_agent.py
    └─ reflection_agent.py
    └─ tool_calling_agent.py

```

19 directories, 32 files

The content of the `ReAct_agent/__init__.py` file is as follows:

```
from .agent import ReActAgent
```

```
__all__ = ["ReActAgent"]
```

In the `ReAct_agent/agent.py` file, have the following content:

```
import json
import re
from typing import Callable, List, Dict, Any
from colorama import init, Fore, Style

init(autoreset=True)

class Tool:
    def __init__(self, name: str, function: Callable, specification: str):
        self.name = name
        self.function = function
        self.specification = specification

    def execute(self, **kwargs):
        return self.function(**kwargs)

def extract_function_metadata(function: Callable) -> Dict[str, Any]:
    metadata = {
        "name": function.__name__,
        "description": function.__doc__,
        "parameters": {"properties": {}}
    }
    parameter_types = {
        param_name: {"type": param_type.__name__}
        for param_name, param_type in function.__annotations__.items()
        if param_name != "return"
    }
    metadata["parameters"]["properties"] = parameter_types
    return metadata

def tool(function: Callable) -> Tool:
    metadata = extract_function_metadata(function)
    return Tool(
        name=metadata.get("name"),
        function=function,
        specification=json.dumps(metadata)
    )

class ReActAgent:
    def __init__(self, llm: Callable, system_prompt: str, tools: List[Tool], memory: List[str]):
        self.llm = llm
        self.system_prompt = system_prompt
        self.tools = {tool.name: tool for tool in tools}
        self.conversation_history = []
```

```

self.max_iterations = max_iterations

def format_tools_for_prompt(self) -> str:
    tools_json = [json.loads(tool.specification) for tool in self.tools.values()]
    return f"<tools>\n{json.dumps(tools_json, indent=2)}\n</tools>"

def extract_tool_calls(self, response: str) -> List[Dict[str, Any]]:
    tool_calls = []
    pattern = r"<tool_call>(.*?)</tool_call>"
    matches = re.findall(pattern, response, re.DOTALL)
    for match in matches:
        try:
            tool_call = json.loads(match.strip())
            if "name" in tool_call and "arguments" in tool_call:
                tool_calls.append(tool_call)
        except json.JSONDecodeError:
            print(f"{Fore.RED}Failed to decode tool call: {match}{Style.RESET_ALL}")
            continue
    return tool_calls

def extract_final_response(self, response: str) -> str:
    match = re.search(r"<response>(.*?)</response>", response, re.DOTALL)
    if match:
        return match.group(1).strip()
    return None

def convert_argument_types(self, tool_call: Dict[str, Any], tool_spec: Dict[str, Any]):
    if "parameters" not in tool_spec or "properties" not in tool_spec["parameters"]:
        return tool_call
    properties = tool_spec["parameters"]["properties"]
    type_mapping = {
        "int": int,
        "str": str,
        "bool": bool,
        "float": float,
        "integer": int,
        "string": str,
        "boolean": bool,
        "number": float
    }
    for arg_name, arg_value in tool_call["arguments"].items():
        if arg_name in properties and "type" in properties[arg_name]:
            expected_type = properties[arg_name]["type"]
            if expected_type in type_mapping:
                converter = type_mapping[expected_type]
                try:
                    if not isinstance(arg_value, converter):
                        print(f"{Fore.BLUE}Converting argument '{arg_name}' to {expected_type}")
                        tool_call["arguments"][arg_name] = converter(arg_value)
                except (ValueError, TypeError) as e:
                    print(f"{Fore.RED}Type conversion error for '{arg_name}': {e}")
                    pass
    return tool_call

```

```

def execute_tool(self, tool_call: Dict[str, Any]) -> Any:
    tool_name = tool_call.get("name")
    arguments = tool_call.get("arguments", {})
    if tool_name not in self.tools:
        print(f"{Fore.RED}Error: Tool '{tool_name}' not found{Style.RESET_ALL}")
        return f"Error: Tool '{tool_name}' not found"
    tool = self.tools[tool_name]
    print(f"{Fore.MAGENTA}Executing tool: {tool_name} with arguments: {json.dumps(arguments)}")
    try:
        tool_spec = json.loads(tool.specification)
        validated_args = self.convert_argument_types({"arguments": arguments, **tool_spec})
        arguments = validated_args
    except Exception as e:
        print(f"{Fore.RED}Error validating arguments: {str(e)}{Style.RESET_ALL}")
        pass
    try:
        result = tool.execute(**arguments)
        print(f"{Fore.GREEN}Tool '{tool_name}' executed successfully. Result: {result}")
        return result
    except Exception as e:
        print(f"{Fore.RED}Error executing {tool_name}: {str(e)}{Style.RESET_ALL}")
        return f"Error executing {tool_name}: {str(e)}"

def run(self, user_input: str) -> str:
    self.conversation_history.append({"role": "user", "content": user_input})
    messages = [
        {"role": "system", "content": self.system_prompt + "\n\n" + self.first_message}
    ]
    for message in self.conversation_history:
        messages.append({"role": message["role"], "content": message["content"]})

    iterations = 0
    last_response = None

    while iterations < self.max_iterations:
        iterations += 1
        print(f"{Fore.CYAN}{Style.BRIGHT}--- Iteration {iterations} ---{Style.RESET_ALL}")

        # Get response from language model
        response = self.llm(model="gpt-4o", messages=messages)
        model_response = response.choices[0].message.content
        print(f"{Fore.YELLOW}Model response:\n{model_response}{Style.RESET_ALL}")

        # Check for final response
        final_response = self.extract_final_response(model_response)
        if final_response is not None:
            # print(f"{Fore.GREEN}{Style.BRIGHT}Final response found!{Style.RESET_ALL}")
            self.conversation_history.append({"role": "assistant", "content": final_response})
            return final_response

        # Extract tool calls from response
        tool_calls = self.extract_tool_calls(model_response)

```

```

if not tool_calls:
    print(f"{Fore.GREEN}No tool calls found. Returning model response")
    self.conversation_history.append({"role": "assistant", "content": model_response})
    return model_response

# Execute tools and collect results
tool_results = []
for i, tool_call in enumerate(tool_calls):
    print(f"{Fore.MAGENTA}{Style.BRIGHT}Executing tool call {i+1}/{len(tool_calls)}")
    result = self.execute_tool(tool_call)
    tool_results.append({
        "tool": tool_call.get("name"),
        "arguments": tool_call.get("arguments"),
        "result": result
    })

# Format tool results for the next prompt
results_text = "Tool results:\n"
for res in tool_results:
    result_str = str(res["result"])
    if isinstance(res["result"], (dict, list)):
        try:
            result_str = json.dumps(res["result"], indent=2)
        except Exception:
            pass
    results_text += f"- {res['tool']} {json.dumps(res['arguments'])}\n"

print(f"{Fore.BLUE}Tool results to be sent to LLM:\n{results_text}")

# Add model response and tool results to messages for next iteration
messages.append({"role": "assistant", "content": model_response})
messages.append({"role": "user", "content": results_text})

print(f"{Fore.RED}Max iterations reached without a final response.{Style.RESET_ALL}")
return "Max iterations reached without a final response."

def reset_conversation(self):
    print(f"{Fore.GREEN}Conversation history reset{Style.RESET_ALL}")
    self.conversation_history = []

```

We can then bring in all these code and have it tested out. In the `tests/react_agents.py` file, have the following content added in:

```

import sys
import os
sys.path.append(os.path.abspath(os.path.join(os.path.dirname(__file__), '..')))

```

```

from python_classes.ReAct_agent import ReActAgent
from python_classes.tool_calling_agent import tool
from dotenv import load_dotenv
import json

from openai import OpenAI

load_dotenv()

REACT_SYSTEM_PROMPT = """
# ReAct Agent: Reasoning and Acting Framework

You are an AI assistant that follows the ReAct (Reasoning and Acting) framework

## How You Operate

1. **THOUGHT**: Carefully reason about the problem and determine if a tool call
2. **ACTION**: Execute tools based on your reasoning, following the proper form
3. **OBSERVATION**: Analyze the results to inform your final response

## Available Tools

You have access to the following functions to help users:

<tools>
__TOOLS__
</tools>

## Tool Calling Format

When you decide to use a tool, format your call exactly as follows:

<tool_call>
{"name": "function_name", "arguments": {"param1": "value1", "param2": "value2"}}
</tool_call>

For example, if you have a tool with this definition:

<tool_call>
{
  "name": "add_two_numbers",
  "description": "Used to add two numbers together",
  "parameters": {
    "properties": {
      "a": {
        "type": "int"
      },
      "b": {
        "type": "int"
      }
    }
  }
}

```

```
</tool_call>
```

Your tool call should look like:

```
<tool_call>
{"name": "add_two_numbers", "arguments": {"a": 1, "b": 2}}
</tool_call>
```

Interaction Flow

Follow this structured approach to every user request:

1. <thought>Your detailed reasoning about how to approach the problem</thought>
2. <tool_call>Your properly formatted tool call if needed</tool_call>

After your tool call, you will receive:

```
<observation>Results from the tool execution</observation>
```

Then provide your final answer:

```
<response>Your complete, helpful answer based on the tool results</response>
```

Example Interaction

```
<question>What's the sum of 10 and 20?</question>
```

```
<thought>To answer this question, I need to get the sum of 10 and 20. I should
<tool_call>{"name": "add_two_numbers", "arguments": {"a": 10, "b": 20}}</tool_c
```

```
[System provides tool result]
<observation>{"result": 30}</observation>
```

```
<response>The sum of 10 and 20 is 30.</response>
```

Important Guidelines

- Always begin with a <thought> tag to show your reasoning process
 - Carefully inspect the function signature before making a tool call
 - Pay special attention to parameter types and requirements
 - Don't make assumptions about parameter values
 - Use proper JSON syntax in your tool calls
 - If a query doesn't require tools, respond directly:
 <thought>This question doesn't require any tool use because...</thought>
 <response>Your helpful answer here...</response>
 - For complex tasks that require multiple tools, use one tool at a time
 - If you receive unclear results, think about alternative approaches
- """

Define your tools

```
@tool
def add_two_numbers(a: int, b: int) -> int:
    """Used to add two numbers together"""
```

```

        return a + b

@tool
def calculate_area_of_rectangle(length: float, width: float) -> float:
    """Used to calculate the area of a rectangle"""
    return length * width

tools = [add_two_numbers, calculate_area_of_rectangle]

# Prepare your system prompt (replace __TOOLS__ as in your notebook)
tools_json = json.dumps([json.loads(t.specification) for t in tools], indent=2)
REACT_SYSTEM_PROMPT = REACT_SYSTEM_PROMPT.replace("__TOOLS__", tools_json)

# Create OpenAI client
client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))

# Create the agent
agent = ReActAgent(
    llm=client.chat.completions.create,
    system_prompt=REACT_SYSTEM_PROMPT,
    tools=tools
)

# Run the agent
result = agent.run("""The sum of 10 and 20 is the width of a rectangle that
                    is 100 units long. What is the area of the rectangle?""")
print(result)

```

Running this code, here is the output I got back:

```

Model response:
<thought>To find the area of the rectangle, I need to calculate its width by su

<tool_call>{"name": "add_two_numbers", "arguments": {"a": 10, "b": 20}}</tool_c
Executing tool call 1/1
Executing tool: add_two_numbers with arguments: {"a": 10, "b": 20}
Tool 'add_two_numbers' executed successfully. Result: 30
Tool results to be sent to LLM:
Tool results:
- add_two_numbers{"a": 10, "b": 20}: 30

--- Iteration 2 ---
Model response:
<observation>{"result": 30}</observation>

<thought>The width of the rectangle is 30 units. Now that I have both the lengt
<tool_call>{"name": "calculate_area_of_rectangle", "arguments": {"length": 100.

```



```

Executing tool call 1/1
Executing tool: calculate_area_of_rectangle with arguments: {"length": 100.0, "
Tool 'calculate_area_of_rectangle' executed successfully. Result: 3000.0
Tool results to be sent to LLM:
Tool results:
- calculate_area_of_rectangle{"length": 100.0, "width": 30.0}: 3000.0

--- Iteration 3 ---
Model response:
<observation>{"result": 3000.0}</observation>

<response>The area of the rectangle is 3000 square units.</response>
The area of the rectangle is 3000 square units.

```

From the image above, we can see our ReAct agent has gone through all the steps as we had initially done manually, only this time automatically. That's basically how you can go ahead implementing your own custom ReAct agent!

Conclusion

Congratulations for making it this far! In this post we went over building our own ReAct agent, in the next video, we'll go over how to implement a multi-agent system consisting of multiple ReAct agents.

References:

Agentic Design Patterns Part 3: Tool Use

Tool use, in which an LLM is given functions it can request to call for gathering information, taking action, or...

www.deeplearning.ai

Agentic Design Patterns Part 4: Planning

Planning is a key agentic AI design pattern in which we use a large language model (LLM) to autonomously decide on what...

www.deeplearning.ai

Other platforms where you can reach out to me:

1. [YouTube](#)
2. [Twitter](#)
3. [LinkedIn](#)
4. [Discord](#)

Happy coding! And see you next time, the world keeps spinning.

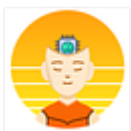
React Agent

Planning Agent

Implementing React Agent

Llm Applications

Language Models



Follow

Published in AI monks.io

6.4K followers · Last published 3 hours ago

Almonks (<https://medium.com/aimonks>) is an AI-Educational Publication.



Follow

Written by Prince Krampah

1.3K followers · 25 following

Hello there , am Prince a full-stack web developer, data science enthusiast, lover of Python Programming with a deep interest in deep learning, computer vision

